

ログイン画面通過後の打刻画面

ログインが成功した直後の状態は、「通行証をもらったけれど、まだ入り口に立っている」状態です。この通行証（ユーザーID）を使って、打刻ボタンを表示し、実際にバックエンドへ打刻データを送る機能を実装しましょう。

基礎的な機能としては何が必要か

ログイン状態の記憶 (State Management)

Vue.js側で、「今誰がログインしているか」を管理する必要があります。

今回はシンプルに、`App.vue`（親玉）がログイン情報を持ち、ログイン前は「ログイン画面」、ログイン後は「打刻画面」と表示を切り替えるようにします。

画面の切り替え (Conditional Rendering)

Vue.jsの `v-if` という機能を使って、ログイン状態に応じて表示するコンポーネントをスイッチします。

ステータスの動的反映と打刻ボタンの実装

新しく `AttendanceBoard.vue` というコンポーネントを作成します。この画面が開かれたとき、バックエンドの `/api/attendance/status` を叩いて、「今出勤中か？」を確認し、ボタンを切り替えます。

実装手順

① frontend/src/components/AttendanceBoard.vue を新規作成

ここにメインの打刻ロジックを詰め込みます。

```
<template>
  <div class="attendance-board">
    <h3>ようこそ、{{ userName }} さん</h3>
    <div class="status-badge" :class="status">
      現在の状態: {{ status === 'CLOCKED_IN' ? '勤務中' : '未出勤' }}
    </div>

    <div class="actions">
      <button v-if="status === 'CLOCKED_OUT'" @click="punch('clock-in')" class="btn-in">出勤</button>
      <button v-if="status === 'CLOCKED_IN'" @click="punch('clock-out')" class="btn-out">退勤</button>
    </div>
  </div>
</template>
```

```
<script setup>
import { ref, onMounted } from 'vue';
import apiClient from '../api';

const props = defineProps(['accountId', 'userName']);
const status = ref('CLOCKED_OUT');

// 画面が開いたときに現在の状態を取得
const fetchStatus = async () => {
  try {
    const response = await apiClient.get(`/attendance/status?accountId=${props.accountId}`);
    status.value = response.data; // "CLOCKED_IN" or "CLOCKED_OUT"
  } catch (error) {
    console.error('ステータス取得失敗', error);
  }
};

// 打刻処理
const punch = async (type) => {
  try {
    await apiClient.post(`/attendance/${type}?accountId=${props.accountId}`);
    alert(`${type} === 'clock-in' ? '出勤' : '退勤'しました！`);
    fetchStatus(); // 打刻後に状態を再取得してボタンを切り替える
  } catch (error) {
    alert('打刻に失敗しました。');
  }
};

onMounted(fetchStatus);
</script>
```

```
<style scoped>
.attendance-board { text-align: center; margin-top: 50px; }
.status-badge { display: inline-block; padding: 10px 20px; border-radius: 20px; margin-bottom: 20px; font-weight: bold; }
.CLOCKED_IN { background-color: #e3f2fd; color: #1976d2; }
.CLOCKED_OUT { background-color: #f5f5f5; color: #616161; }
.btn-in { background-color: #4caf50; color: white; padding: 15px 30px; font-size: 1.2rem; border: none; border-radius: 5px; cursor: pointer; }
.btn-out { background-color: #f44336; color: white; padding: 15px 30px; font-size: 1.2rem; border: none; border-radius: 5px; cursor: pointer; }
</style>
```

② frontend/src/App.vue を修正して画面遷移を作る

App.vue を「ログイン画面」と「打刻画面」の司令塔にします。

```
<template>
  <div id="app">
    <LoginForm v-if="!user" @login-success="handleLoginSuccess" />

    <AttendanceBoard v-else :accountId="user.accountId" :userName="user.userName" />
  </div>
</template>
```

```
<script setup>
import { ref } from 'vue';
import LoginForm from './components/LoginForm.vue';
import AttendanceBoard from './components/AttendanceBoard.vue';

const user = ref(null);

const handleLoginSuccess = (userData) => {
  user.value = userData; // サーバーから返ってきたユーザー情報を保持
};
</script>
```

③ LoginForm.vue を少し修正

ログイン成功時に、親（App.vue）へデータを渡すようにします。

```
const handleLogin = async () => {
  try {
    const response = await apiClient.post('/users/login', {
      userId: userId.value,
      password: password.value
    });

    emit('login-success', response.data); // alertは消して、親コンポーネントにイベントを送る
  } catch (error) {
    console.error('ログイン失敗:', error);
    errorMessage.value = 'ユーザーIDまたはパスワードが正しくありません。';
  }
};

// script setup の冒頭に emit の定義を追加
const emit = defineEmits(['login-success']);
```

💡 この実装のポイント

1. `onMounted` : 画面（コンポーネント）が表示された瞬間に実行される処理です。ここで「今、出勤してるっけ？」とバックエンドに聞きに行きます。
2. `v-if / v-else` : これにより、URLを変えなくても「ログイン前」と「ログイン後」の画面をパッと切り替えることができます（SPA: Single Page Application の基本です）。
3. **状態の再取得**: 出勤ボタンを押した直後に `fetchStatus()` をもう一度呼ぶことで、画面をリロードしなくてもボタンが「退勤」に変わるようになります。

まずはこの `AttendanceBoard.vue` を作成し、画面がパッと切り替わって「出勤ボタン」が出てくるか試してみてください！

出勤 / 退勤のステータスを可変にするには

「出勤/退勤」だけでなく「入室/退室」や「出席/退席」など、用途に合わせてラベルを可変にする ことで、このアプリの実用性はさらに増します。

これを実現するには、「ラベル設定（文言）」をデータとして持たせ、それを画面側に反映させる 仕組みを導入する必要があります。

拡張性を考慮した、3つのステップで修正案を提示しました。

1. フロントエンド：ラベル管理の導入

AttendanceBoard.vue のハードコードされている「出勤」「退勤」という文字を、変数に置き換えます。

```
<template>
  <div class="attendance-board">
    <h3>ようこそ、{{ userName }} さん</h3>

    <div class="status-badge" :class="status">
      現在の状態: {{ status === 'CLOCKED_IN' ? labels.inActive : labels.outActive }}
    </div>

    <div class="actions">
      <button v-if="status === 'CLOCKED_OUT'" @click="punch('clock-in')" class="btn-in">
        {{ labels.inAction }}
      </button>
      <button v-if="status === 'CLOCKED_IN'" @click="punch('clock-out')" class="btn-out">
        {{ labels.outAction }}
      </button>
    </div>
  </div>
</template>
```

```
<script setup>
import { ref, onMounted, computed } from 'vue';
// ... (apiClientのインポートなど)

const props = defineProps(['accountId', 'userName', 'mode']); // mode プロパティを追加

// モードに応じたラベルの定義
const labelSettings = {
  attendance: { inActive: '勤務中', outActive: '未出勤', inAction: '出勤', outAction: '退勤' },
  room:       { inActive: '入室中', outActive: '退室済', inAction: '入室', outAction: '退室' },
  session:    { inActive: '出席中', outActive: '退席中', inAction: '出席', outAction: '退席' }
};

// 現在のモード (propsから受け取る。デフォルトは 'attendance')
const labels = computed(() => labelSettings[props.mode || 'attendance']);

// ... (fetchStatus や punch メソッドはそのまま)
</script>
```

2. バックエンド：ステータス管理の汎用化

現在は `CLOCKED_IN / CLOCKED_OUT` という名前を使っていますが、意味合いとしてより汎用的な `ACTIVE / INACTIVE`（有効/無効）や、単に `IN / OUT` という内部状態として扱うのが望ましいです。

Java側の `Enum` やデータベースの値を書き換えるのは少し手間がかかるため、まずは「内部的な状態（IN/OUT）」と「画面上の見せ方（表示名）」を切り離して考えるのがコツです。

3. 親コンポーネントでの切り替え

`App.vue` などで、このアプリを「どのモード」で使いたいかを指定できるようにします。

```
<AttendanceBoard  
  v-else  
  :accountId="user.accountId"  
  :userName="user.userName"  
  mode="room"  
>
```

※ `mode="room"` に変えるだけで、画面上のボタンが「入室」「退室」に切り替わります。

さらに進化させるなら：DBにラベルを持たせる

もし、「ユーザーごとに好きなラベルを設定したい」場合は、以下の構成に変更します。

1. **データベース**: `USER_ACCOUNTS` テーブルに `label_in_name` (例: "登校") と `label_out_name` (例: "下校") というカラムを追加。
2. **API**: ログイン時にこれらのラベル名も一緒に返却する。
3. **フロントエンド**: 受け取ったラベル名をボタンに表示する。

こうすることで、システム自体を一切書き換えることなく、管理画面から「塾用」「オフィス用」「イベント用」とカスタマイズできる**マルチユースな勤怠管理システム**へと進化します。

まずは上記の「フロントエンドでのラベル切り替え」を試してみて感触を確かめてみてください。